

CS & IT ENGINEERING



Computer Network - 2

Framing & Error Control

Lecture No. - 02

By - Abhishek Sir





Recap of Previous Lecture



Topic

Framing

- ① Byte(char) count
- ② Byte(char) stuffing
(SFD & EFD)





Topics to be Covered



Topic

Bit Stuffing

Topic

Two-dimension (2D) Parity



Topic : Bit Stuffing



Replace "**STX**" and "**ETX**" with "**FLAG**" character.

↓
FLAG : "~"

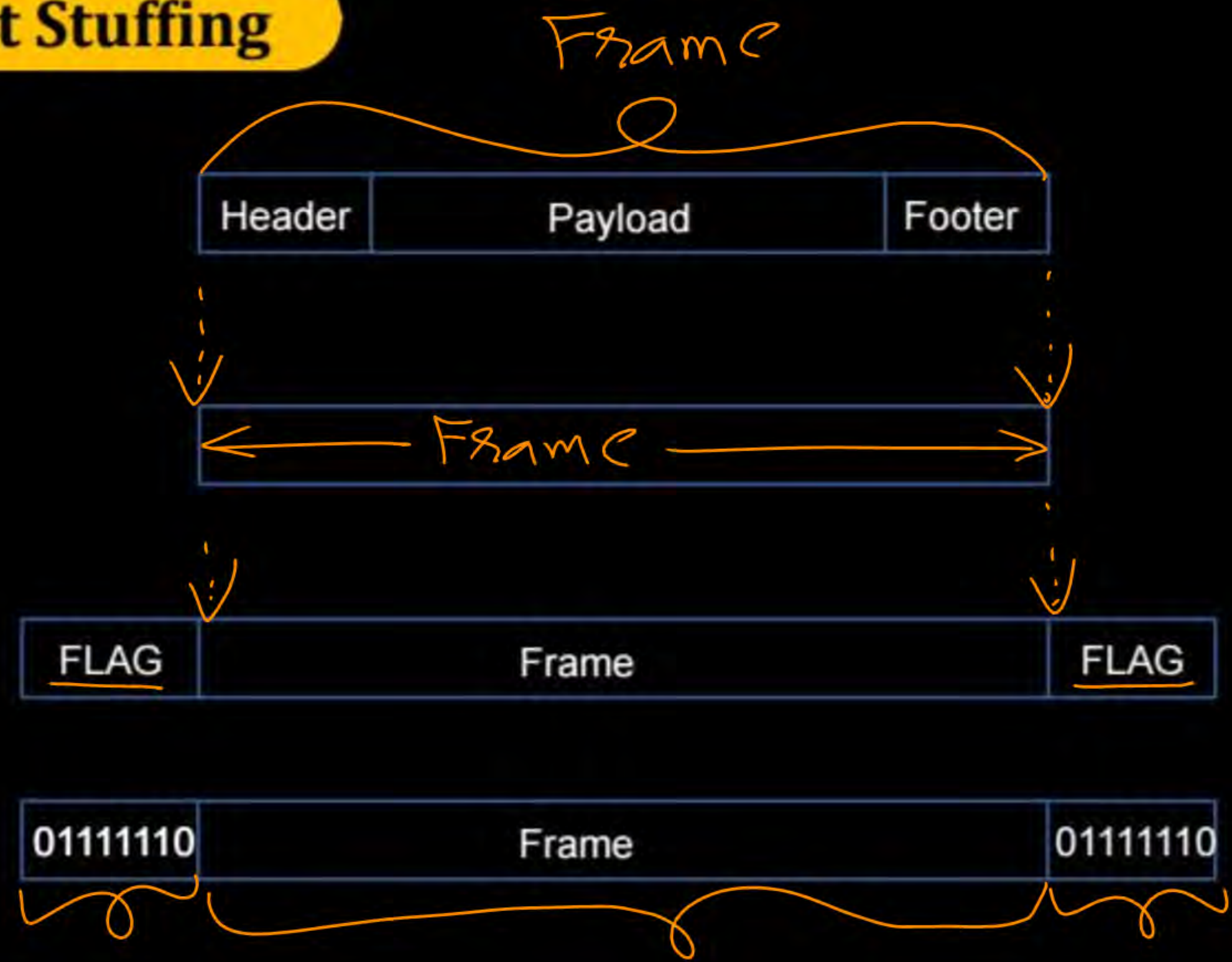
Binary (bit pattern) = "01111110"

ASCII Value = 126 = 0x7E

[FLAG ← SFD, EFD]



Topic : Bit Stuffing





Topic : Bit Stuffing





Topic : Bit Stuffing



Transmitter Protocol :

1. Transmit "**FLAG**" character, just before frame transmission start.
2. Transmit "**FLAG**" character, just after frame transmission completed.
3. Stuff "**Zero Bit**" after every **five continuous one's** while transmission. *of frame.*
[Except start and end of frame]

↓
No stuffing



Topic : Bit Stuffing



Receiver Protocol :

1. Looking for “Start of Frame” [**FLAG**].
2. After Start of Frame, it is looking for “End of Frame” [**FLAG**].
- 3. Discard “Zero Bit” that found after every five continuous one's while receiving data frame.



Topic : Bit Stuffing



CASE I:

Frame 1 = 11011101110101

Transmitted Data =

011111101101110111010101111110



Topic : Bit Stuffing



CASE I :

Frame 1 = 11011101110101

Output = 01111110 11011101110101 01111110

↑ SFD ↓ EFD



C_1 C_2

8 bit
 0111110

01111110 10111110 1010111110 (Incorrect)

↑ SFD
↓ EFD



Transmitted Data =

(correct)

Frame = 101111110101 | 01111110



Topic : Bit Stuffing

[Output = Input with bit stuffing] 

CASE II :

Frame 2 = 10111110101
Input

Output = 01111110 10111110 1010101 01111110

Output



Frame 3 = 10111101010

Transmitted Data =

0111111010111110101000111110

↑

↓
discard

Recv := 10111111010 01111110

Frame EFD



Topic : Bit Stuffing



CASE III: (correct)

Frame 3 = 10111101010

Transmitted data =

0111110101111001010011110

↑
discarded

correct

Recv:-

Frame = 10111101010 0111110

Frame EFD



Topic : Bit Stuffing



CASE III :

Frame 3 = 10111101010

Output = 01111110 1 0 1 1 1 1 1 0 1 0 1 0 01111110



Topic : Bit Stuffing



CASE IV :

Frame 4 = 111110111101

Transmitted Data =

0111111011111001111010111110



Topic : Bit Stuffing



CASE IV :

Frame 4 = 111110111101

Output = 01111110 1111100111101 01111110



Topic : Bit Stuffing



Conclusion :

Six continuous one's can only appear in the start and end of frame while transmission.

Advantage :

For every five continuous one's present in the data frame, length of frame increases by "one-bit".

Bit stuffing - (Recv) AT



0111111

→ 0 (discard)

→ 1

→ 0

EFD

01111110

→ 1

Seven one

Error

[MCQ]

IIT-KGP, H.W.

[GATE-2014][1 Mark]



#Q. A bit-stuffing based framing protocol uses an 8-bit delimiter pattern of 01111110. If the output bit-string after stuffing is 01111100101, then the input bit-string is :

- A 0111110100
- B 0111110101
- C 0111111101
- D 0111111111

[MCQ]

IIT-D, H.W.

[GATE-IT-2004] [1 Mark]



#Q. In a data link protocol, the frame delimiter flag is given by 0111. Assuming that bit stuffing is employed, the transmitter sends the data sequence 01110110 as:

FLAG = 01110

- A 01101011
- B 011010110
- C 011101100
- D 0110101100

0111 | 01010100111
01110 | 01010100110



Topic : Error Detection and Correction



Types of error correction techniques :

1. Two-dimension (2D) Parity
2. Hamming Code

one-bit parity }
CRC }
↓
VRC

checksum }
2D parity }
↓
LRC



Topic : Two-dimension (2D) Parity



→ Both sender and receiver must agree on same block size and same parity (either even or odd parity)

→ Block Size = m * n
[m rows & n columns]

Number of Data bits = $(m-1) * (n-1)$

Number of parity bits = $[m + n - 1]$





Topic : Two-dimension (2D) Parity



Suppose Block Size = 4 x 6 and "Even Parity"

Data = "d₁d₂d₃d₄d₅ d₆d₇d₈d₉d₁₀ d₁₁d₁₂d₁₃d₁₄d₁₅"

Sender

d ₁	d ₂	d ₃	d ₄	d ₅
d ₆	d ₇	d ₈	d ₉	d ₁₀
d ₁₁	d ₁₂	d ₁₃	d ₁₄	d ₁₅



Topic : Two-dimension (2D) Parity



Suppose **Block Size = 4 x 6** and **“Even Parity”**

Data = “ $d_1d_2d_3d_4d_5 d_6d_7d_8d_9d_{10} d_{11}d_{12}d_{13}d_{14}d_{15}$ ”

Sender

d_1	d_2	d_3	d_4	d_5	R_1
d_6	d_7	d_8	d_9	d_{10}	R_2
d_{11}	d_{12}	d_{13}	d_{14}	d_{15}	R_3
C_1	C_2	C_3	C_4	C_5	P

↓
Parity
of Parity



Topic : Two-dimension (2D) Parity



$$R_1 = d_1 \oplus d_2 \oplus d_3 \oplus d_4 \oplus d_5$$

$$R_2 = d_6 \oplus d_7 \oplus d_8 \oplus d_9 \oplus d_{10}$$

$$R_3 = d_{11} \oplus d_{12} \oplus d_{13} \oplus d_{14} \oplus d_{15}$$

$$C_1 = d_1 \oplus d_6 \oplus d_{11}$$

$$C_2 = d_2 \oplus d_7 \oplus d_{12}$$

$$C_3 = d_3 \oplus d_8 \oplus d_{13}$$

$$C_4 = d_4 \oplus d_9 \oplus d_{14}$$

$$C_5 = d_5 \oplus d_{10} \oplus d_{15}$$

Suppose Block Size = 4 x 6 and "Even Parity"

Data = "d₁d₂d₃d₄d₅ d₆d₇d₈d₉d₁₀ d₁₁d₁₂d₁₃d₁₄d₁₅"

Sender

d ₁	d ₂	d ₃	d ₄	d ₅	R ₁
d ₆	d ₇	d ₈	d ₉	d ₁₀	R ₂
d ₁₁	d ₁₂	d ₁₃	d ₁₄	d ₁₅	R ₃
C ₁	C ₂	C ₃	C ₄	C ₅	P

$$P = R_1 \oplus R_2 \oplus R_3 = C_1 \oplus C_2 \oplus C_3 \oplus C_4 \oplus C_5$$



Topic : Two-dimension (2D) Parity



Suppose **Block Size** = **4 x 6** and “**Even Parity**”

Data = “ $d_1d_2d_3d_4d_5$ $d_6d_7d_8d_9d_{10}$ $d_{11}d_{12}d_{13}d_{14}d_{15}$ ”

Sender

d_1	d_2	d_3	d_4	d_5	R_1
d_6	d_7	d_8	d_9	d_{10}	R_2
d_{11}	d_{12}	d_{13}	d_{14}	d_{15}	R_3
C_1	C_2	C_3	C_4	C_5	P

Transmitted Data = “ $d_1d_2d_3d_4d_5R_1$ $d_6d_7d_8d_9d_{10}R_2$ $d_{11}d_{12}d_{13}d_{14}d_{15}R_3$ $C_1C_2C_3C_4C_5P$ ”

codeword / valid code



Topic : Two-dimension (2D) Parity



Suppose **Block Size = 4 x 6** and "Even Parity"

Data = "1 0 0 1 1 0 1 1 0 1 1 0 0 1 0"

Sender					
1	0	0	1	1	1
0	1	1	0	1	1
1	0	0	1	0	0
0	1	1	0	0	0



Topic : Two-dimension (2D) Parity



Suppose Block Size = 4 x 6 and "Even Parity"

Data = "1 0 0 1 1 0 1 1 0 1 1 0 0 1 0"

(Sender)

1	0	0	1	1	1
0	1	1	0	1	1
1	0	0	1	0	0
0	1	1	0	0	0

Transmitted Data = "1 0 0 1 1 1 0 1 1 0 1 1 1 0 0 0 0 1 1 0 0 0"

Valid Code



Topic : Two-dimension (2D) Parity



CASE I : No any error

Suppose Block Size = 4 x 6 and "Even Parity"

Receiver

1	0	0	1	1	1	✓
0	1	1	0	1	1	✓
1	0	0	1	0	0	✓
0	1	1	0	0	0	✓
✓	✓	✓	✓	✓	✓	

Transmitted Data = "1 0 0 1 1 1 0 1 1 0 1 1 1 0 0 1 0 0 0 1 1 0 0 0"

Received Data = "1 0 0 1 1 1 0 1 1 0 1 1 1 0 0 1 0 0 0 1 1 0 0 0"

Receiver Concluded : No any error detected



Topic : Two-dimension (2D) Parity



if receiver finds all row wise and column wise parities are balanced
then receiver concluded "No any error detected"

else

receiver concluded "Error detected"



2 mins Summary



Topic

Bit Stuffing ✓

Topic

Two-dimension (2D) Parity →



THANK - YOU

